Философия и инженерное мышление в Go

Почему возник Go

- Сентябрь 2007 года. Google. Роб Пайк ждёт, пока скомпилируется C++ проект - 45 минут! В этот момент он думает не о синтаксисе нового языка, а о том, что вся модель разработки масштабного ПО уже сломана!

- Три человека - Rob Pike, Ken Thompson и Robert Griesemer начинают проектировать язык.
  - Ни одна идея не попадала в спецификацию, пока все трое не соглашались, что она верна
  - Griesemer позже заметил, что язык получился лучше, чем тот, который каждый из них спроектировал бы в одиночку

Проблема, которую решает Go

- Go - не ответ на вопрос «Как сделать язык мощнее?».
  - Go - ответ на вопрос «Как инженерить программное обеспечение при масштабе Google?»
- Конкретные боли:
  - Время сборки - десятки миллионов строк C++, часы компиляции даже на распределённых кластерах
  - Управление зависимостями - #include в C/C++ механизм включения исходного кода, а не модульности. Каскадные пересборки неизбежны
  - Когнитивная нагрузка - C++ стал языком, в котором нужно знать язык, чтобы читать язык. Спецификация C++20 — ~1800 страниц. Спецификация Go — ~100 страниц
  - Команды, не гении - код в Google пишут тысячи инженеров. Язык должен быть понятен среднему разработчику в команде, а не только автору

Проблема, которую решает Go

«Наши программисты — не исследователи. Они, как правило, молоды, после университета. Язык, который мы им даём, должен быть лёгким для понимания и освоения»

Rob Pike

Ретроспектива

- Программирование 70-90х отличается от 0-ых
  - Программист понимал, что стоит за каждой строкой кода. Аллокация - конкретный вызов. Данные - конкретная структура в памяти. Цена каждого решения - прозрачна.
- 2000-е уже эра абстракций
  - ООП обещало структурировать сложность - и породило новую сложность (паттерны как лекарство от паттернов)
  - Виртуальные машины скрыли аппаратную платформу
  - Прототипы на «высокоуровневых» языках превращались в production без рефакторинга
  - Мы потеряли понимание того, как наш код взаимодействует с железом

Как мы «решали» проблемы?

- Медленно работает?
  - Добавим больше серверов
- Сложно поддерживать?
  - Добавим больше разработчиков
- Обе стратегии линейно увеличивают стоимость при экспоненциальном росте сложности

Как мы «решали» проблемы?

Надежда на то, что прогресс аппаратного обеспечения излечит все болезни ПО, не оправдалась. Критический наблюдатель заметит, что софт умудряется обгонять железо по размеру и медлительности

Niklaus Wirth

Самое поразительное достижение индустрии ПО - её непрерывная нейтрализация устойчивого и ошеломительного прогресса аппаратной индустрии

Henry Petroski

Философия проектирования

- Иерархия приоритетов при код-ревью и принятии инженерных решений:
  - Целостность
  - Читабельность)
  - Простота
  - Производительность

- Вы должны осознанно и обоснованно объяснить, в какой категории принимаете решение. Если вы оптимизируете производительность в ущерб целостности - вы должны это знать и аргументировать!

Целостность

- Микроуровень
  - Каждая аллокация, каждое чтение и запись в память - точны, консистентны и эффективны. Система типов - критический инструмент обеспечения этого уровня целостности
- Макроуровень
  - Каждая трансформация данных - точна, консистентна и эффективна. Минимизация кода и обработка ошибок — инструменты обеспечения макро-целостности

Почему это на первом месте?

- Исследования показывают, что количество багов пропорционально количеству строк кода.
  - Не сложности алгоритмов, не «качеству» разработчиков — количеству строк. Каждая строка — потенциальный баг. Каждая необработанная ошибка — нарушение целостности.

Почему это на первом месте?

- В Go это проявляется фундаментально:
  - Явная обработка ошибок вместо исключений — вы не можете «забыть» об ошибке
  - Строгая система типов без неявных преобразований
  - Нулевые значения (zero values) для каждого типа — нет неинициализированной памяти
- Исключения в Java/C# позволяют «пробросить проблему вверх по стеку, пока не станет слишком поздно её диагностировать»

```go
f, err := os.Open("config.yaml")
if err != nil {
    return fmt.Errorf("opening config: %w", err)
}
defer f.Close()
```

Читабельность

Индустрия ПО - одно из немногих мест, где мы учим людей писать до того, как научим их читать

Tom Love (создатель Objective-C)

Код читается намного больше раз, чем пишется

Dave Cheney

Читабельность ≠ Краткость

Читабельность - это способность среднего разработчика в команде понять, что делает код, без ментального истощения. Не эксперта. Не автора. Среднего разработчика

«Эксперты опускаются вниз. Начинающие поднимаются вверх. Мы встречаемся посередине»

Не скрывайте стоимость

- Каждая строка кода не должна скрывать стоимость того, что она делает
- Как только вы начинаете скрывать стоимость - вы теряете способность смотреть на строку кода и принимать инженерное решение.
- Если вы понимаете стоимость - вы инженерите.
- Если не понимаете - вы просто программируете.

Конец войнам стилей

- Go решил проблему форматирования радикально - один формат для всего кода в мире
- gofmt - не рекомендация, а инструмент, который делает весь Go-код визуально единообразным. В Google код, не прошедший gofmt, отклоняется при коммите
- В итоге вы можете открыть любой Go-проект на GitHub и сразу читать его. Нет когнитивной нагрузки на «привыкание к стилю»

Простота

Простота - великая добродетель, но она требует тяжёлой работы для достижения и образования для оценки. И что ещё хуже: сложность продаётся лучше

Edsger Dijkstra

Простота - это процесс редукции, но чтобы сделать вещи проще, приходится добавлять обёртки - что парадоксально

Rob Pike

Простота ≠ Примитивность

- Простота в Go - осознанное ограничение пространства решений:
  - Нет наследования → Композиция через встраивание (embedding) и интерфейсы
  - Нет перегрузки функций → Одно имя = одно поведение
  - Нет исключений → Ошибки как значения
  - Нет дженериков (до 1.18 ☺) → Конкретный код вместо абстрактного
  - Один цикл for → Вместо for, while, do-while, foreach
  - Нет тернарного оператора → Явный if-else
  - 25 ключевых слов → C++ имеет ~90+

- Когда есть только один способ сделать что-то - весь код выглядит одинаково. А одинаково выглядящий код - читабельный код

Производительность

Производительность стоит на последнем месте. Не потому, что она не важна - а потому что преждевременная оптимизация разрушает все три предыдущих столпа

Сделай правильно, сделай ясно, сделай лаконично, сделай быстро. В этом порядке.

Wes Dyer

Joe Armstrong (создатель Erlang): Сначала заставь работать, потом сделай красиво, и только потом, если действительно, действительно нужно, - быстро. В 90% случаев, если вы сделали красиво, уже быстро.

Joe Armstrong (создатель Erlang)

Производительность

При этом Go спроектирован так, что идиоматический код уже производителен:

- Компиляция в нативный код - никакой VM, никакого JIT warm-up
- Горутины - стоимость ~2-8 KB стека (vs ~1 MB для OS-потока)
- Контигуальные структуры в памяти — []User это непрерывный блок памяти, не массив указателей
- Escape analysis - компилятор решает, где разместить переменную - на стеке или в куче
- GC с минимальными паузами

Производительность

```go
// Структура данных, дружелюбная к кэшу процессора
type Transform struct {
    X, Y, Z float64  // 24 байта, выровнено, контигуально
}

// Слайс трансформов - непрерывный блок в памяти
// CPU prefetcher может предсказать доступ
transforms := make([]Transform, 10000)

// vs Java ArrayList<Transform> - массив указателей на
разбросанные объекты в куче ☺
```

Ментальная модель

Одна из множества проблем мышления - «когнитивная нагрузка»: количество вещей, на которых мы можем сосредоточиться одновременно. Клише - 7±2, но для многих вещей это ещё меньше

Alan Kay

Ментальная модель (ясная модель кода)

- Поддерживать понимание кода — это ключевая задача разработчика.
- Если мы не можем вспомнить, где находится определенная логика или как что-то работает, это сигнал к рефакторингу.
- Структурирование кода так, чтобы он предоставлял наилучшую возможную “ментальную модель”, помогает улучшить понимание.

Числа, которые нужно знать

- ~10 000 строк - объём кода, ментальную модель которого один разработчик может удерживать в голове
- ~1 000 000 строк → нужна команда ~100 человек с постоянной координацией
- Tom Love: «вероятность успеха проекта на миллион+ строк в США - ниже 50%»
- Tom Love: «100 тысяч строк кода помещаются в одну коробку бумаги для принтера»
- В LLM влезает 20 000 – 30 000 строк ☺

Отладка и ментальная модель

Самые трудные баги - те, при которых ваша ментальная модель ситуации просто неверна, и вы не можете увидеть проблему вовсе.

Brian Kernighan

Все знают, что отладка вдвое сложнее написания кода. Значит, если вы пишете настолько умный код, насколько можете, - как вы его будете отлаживать?

Brian Kernighan

Практические следствия для Go

- Go активно помогает поддерживать ментальную модель:
  - Одна функция = одна ответственность, типичная длина 15-25 строк
  - Пакеты как единица ментальной модели - пакет инкапсулирует одну концепцию
  - Явный поток управления - никаких скрытых вызовов конструкторов/деструкторов, никакой магии аннотаций
  - go vet, staticcheck - инструменты ловят то, что ваша ментальная модель может упустить

Рефакторинг как непрерывный процесс

Если вы пишете что-то, чтобы "закончить" — вы уже пишете legacy-код. Если вы не рефакторите код каждые ~6 недель - вы пишете legacy-код

Bill Kennedy

Есть два типа программных проектов: те, что провалились, и те, что превратились в ужасы наследия.

Peter Weinberger

Мы думаем, что ужасный код пишут ужасные разработчики. Но в реальности его пишут разумные разработчики в ужасных обстоятельствах

Sarah Mei

Рефакторинг в Go

- Go делает рефакторинг дешёвым:
  - Быстрая компиляция - цикл обратной связи в секунды
  - Строгая типизация - компилятор ловит большинство ошибок при перестройке
  - gorename, gomvpkg - инструменты безопасного переименования
  - Тестовая инфраструктура встроена - go test из коробки
  - Нет скрытых зависимостей - если что-то не импортировано, его нет

Data-oriented design

Каждая проблема, которую вы решаете - это проблема трансформации данных. Каждая функция, которую вы пишете, берёт входные данные и производит выходные данные

Bill Kennedy

value semantics vs pointer semantics

- Одно из самых глубоких решений, которое Go-разработчик принимает для каждого типа:
  - Value Semantics (семантика значений):
    - Данные копируются при передаче
    - Каждая функция работает со своей копией
    - Безопасно для конкурентности по умолчанию
    - Данные остаются на стеке → меньше давления на GC
  - Pointer Semantics (семантика указателей):
    - Данные разделяются через указатель
    - Одна копия, множество читателей/писателей
    - Эффективно для больших структур
    - Требует координации при конкурентном доступе

value semantics vs pointer semantics

```go
// Value semantics — тип Time из стандартной библиотеки
// Маленький (24 байта), неизменяемый, копирование дёшево
func (t Time) Add(d Duration) Time {
    return Time{...} // возвращает новое значение
}


// Pointer semantics — тип File
// Представляет разделяемый ресурс ОС, копирование бессмысленно
func (f *File) Read(b []byte) (n int, err error) {
    // мутирует состояние через указатель
}
```

Интерфейсы. Композиция вместо иерархии

- Go сознательно отказался от наследования.
  - Rob Pike: *«Я не фанат программирования, управляемого типами, иерархий типов и наследования. Этот подход продвигает важные решения слишком рано в фазу проектирования, до того, как опыт может повлиять на них»*

Интерфейсы. Композиция вместо иерархии

```go
// Интерфейс в Go — это поведение, не иерархия
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// Композиция интерфейсов
type ReadWriter interface {
    Reader
    Writer
}
// Любой тип, реализующий метод Read — уже Reader
// Никакого "implements", никакого coupling к иерархии
```

- Интерфейсы в Go описывают поведение, которое нужно вызывающему коду, а не наследование, выбранное автором типа.
- Это фундаментальный сдвиг от «я наследуюсь от X» к «мне нужен тот, кто умеет X

Горутины и CSP. Не параллелизм, а конкурентность

Конкурентность - не параллелизм. Конкурентность - это структура, параллелизм - это выполнение. Конкурентность - это способ структурировать программу как композицию независимо выполняющихся процессов

Rob Pike

Go строит модель конкурентности на CSP (Communicating Sequential Processes) - идеях Tony Hoare 1978 года

Горутины и CSP. Не параллелизм, а конкурентность

```go
// Горутина — не поток ОС. Это задача, управляемая Go runtime.
// Стоимость создания: ~2-8 KB (vs ~1-8 MB для OS thread)
go processOrder(order)

// Каналы — примитив синхронизации и коммуникации
ch := make(chan Order, 100) // буферизированный канал

// Producer
go func() {
    for _, o := range orders {
        ch <- o  // отправка — может блокировать
    }
    close(ch)
}()

// Consumer
for order := range ch {
    process(order)  // чтение — блокирует до получения данных
}
```

Планировщик Go

- Go runtime содержит свой планировщик (M:N scheduling):
  - G (Goroutine) → логическая задача
  - M (Machine) → поток ОС
  - P (Processor) → контекст выполнения, привязанный к ядру
- Тысячи горутин мультиплексируются на несколько потоков ОС.
- Переключение контекста горутин - наносекунды (vs микросекунды для потоков ОС), потому что оно происходит в userspace, а не через kernel.

Package-oriented design. Пакет - единица дизайна

- В Go нет классов
- Единица организации – пакет
- Есть рекомендации для разных слоёв:
  - Пакеты приложения (app layer):
    - Могут паниковать
    - Оборачивают ошибки контекстом, если не обрабатывают
    - Большинство обработки ошибок — здесь
    - Фокус на интеграционном тестировании

Package-oriented design. Пакет - единица дизайна

- В Go нет классов
- Единица организации – пакет
- Есть рекомендации для разных слоёв:
  - Бизнес-пакеты (business layer):
    - НЕ могут паниковать
    - Оборачивают ошибки контекстом
    - Меньшинство обработки ошибок
    - Фокус на юнит-тестировании

Package-oriented design. Пакет - единица дизайна

- В Go нет классов
- Единица организации – пакет
- Есть рекомендации для разных слоёв:
  - Фундаментальные пакеты (foundation layer):
    - НЕ могут паниковать
    - НЕ оборачивают ошибки
    - Используют стандартный пакет testing
    - Тесты внутри пакета

Структура проекта

```
project/
├── cmd/                # Точки входа (main packages)
│   └── api/
│       └── main.go
├── internal/           # Бизнес-логика (недоступна извне модуля)
│   ├── user/
│   │   ├── user.go
│   │   └── user_test.go
│   └── order/
├── pkg/                # Публичные библиотеки (опционально)
└── foundation/         # Фундаментальные утилиты
    ├── logger/
    └── web/
```

Встроенная инженерная культура

- Go уникален тем, что инженерные практики встроены в тулчейн, а не являются опциональными:

Инструмент	Назначение	Философский принцип
go fmt	Единый стиль кода	Читабельность - нет войн стилей
go vet	Статический анализ	Целостность - ловит ошибки до runtime
go test	Тестирование + бенчмарки	Корректность - тесты не опция
go test -race	Детектор гонок	Целостность - конкурентность безопасна
go build	Компиляция в один бинарник	Простота - нет runtime-зависимостей
go mod	Управление зависимостями	Целостность - воспроизводимые сборки
go doc	Документация из кода	Читабельность - документация рядом
go generate	Кодогенерация	Простота - явная генерация vs магия
pprof	Профилирование CPU/Mem	Производительность - измеряй, не угадывай

Рекомендуемые ресурсы

- Обязательное чтение
  - Effective Go — официальное руководство по идиоматическому Go
  - Go Proverbs — Rob Pike, фундаментальные принципы
  - Go at Google: Language Design in the Service of Software Engineering — Rob Pike, SPLASH 2012
- Bill Kennedy / Ardan Labs
  - Ultimate Go Notebook — Глубочайший разбор механик языка
  - Design Philosophy on Data and Semantics
  - Design Philosophy on Packaging
  - GoTime #172: Design Philosophy
- Доклады
  - Rob Pike — Simplicity is Complicated
  - Rob Pike — Concurrency Is Not Parallelism

  - **Scott Meyers: Cpu Caches and Why You Care**

Go Proverbs - принципы языка от создателя

- Конкурентность и коммуникация:
  - *«Don't communicate by sharing memory, share memory by communicating»*
- В Java/C++ вы создаёте shared memory и защищаете её мьютексами. В Go вы передаёте владение данными через каналы. Данные в один момент времени принадлежат одной горутине - гонки данных невозможны по дизайну, а не по дисциплине кода

Go Proverbs - принципы языка от создателя

- *«Concurrency is not parallelism»*
  - Конкурентность - это структура программы (как вы организуете независимые задачи). Параллелизм - это выполнение (одновременная работа на нескольких ядрах). Можно иметь конкурентную программу на одном ядре. Go даёт вам инструменты структурирования - а планировщик сам решает, параллелить ли
- *«Channels orchestrate; mutexes serialize»*
  - Каналы - для координации потока работы между горутинами. Мьютексы - для защиты доступа к разделяемому состоянию. Это не взаимозаменяемые инструменты, у каждого своя роль

Go Proverbs - принципы языка от создателя

- *«The bigger the interface, the weaker the abstraction»*

- В Java принято создавать интерфейсы с десятками методов. В Go культура - маленькие интерфейсы. io.Reader - один метод. io.Writer - один метод. fmt.Stringer - один метод. Чем меньше интерфейс, тем больше типов его удовлетворяют, тем больше компонентов можно собрать вместе.

Go Proverbs - принципы языка от создателя

- *«Make the zero value useful.»*
- Когда вы объявляете переменную без инициализации - она получает нулевое значение. Go требует, чтобы это нулевое значение было осмысленным

```go
// sync.Mutex — не нужен конструктор, нулевое значение = разблокированный мьютекс
var mu sync.Mutex
mu.Lock() // работает без инициализации

// bytes.Buffer — нулевое значение = пустой буфер, готовый к записи
var buf bytes.Buffer
buf.WriteString("hello") // работает без инициализации

// Ваши типы тоже должны следовать этому принципу:
type Server struct {
    Addr    string        // "" = будет слушать на :8080 (дефолт)
    Timeout time.Duration // 0 = без таймаута или разумный дефолт
}
```

Go Proverbs - принципы языка от создателя

- *«interface{} says nothing.»*

- *Пустой интерфейс (interface{}, теперь any) принимает что угодно, но не даёт никаких гарантий. Это эквивалент Object в Java - признание поражения системы типов. Используйте конкретные интерфейсы с минимумом методов*

Go Proverbs - принципы языка от создателя

- *«Gofmt's style is no one's favorite, yet gofmt is everyone's favorite.»*
- Никто не считает стиль gofmt идеальным. Но все счастливы, что споры о стиле закончились навсегда. Один стиль для всех Go-проектов в мире. Это экономит тысячи часов код-ревью.
- *«A little copying is better than a little dependency.»*
- Это контринтуитивно для разработчиков из экосистемы npm/Maven. В Go лучше скопировать 20 строк утилитарного кода, чем тащить зависимость, которая потянет транзитивные зависимости, может сломаться при обновлении, усложнит аудит безопасности.

Go Proverbs - принципы языка от создателя

- *«Clear is better than clever.»*

- Есть языки, которые прославляют хитрость (Perl, Haskell, порой Rust). Go прославляет ясность. Если код требует комментария «тут мы делаем хитрый трюк» — перепишите его так, чтобы трюк не был нужен.

- *«Don't just check errors, handle them gracefully.»*

- Начинающие Go-разработчики пишут if err != nil { return err } механически. Это проверка, но не обработка. Обработка — это решение: повторить? вернуть дефолт? залогировать и продолжить? обернуть контекстом?

Go - язык облачной инфраструктуры

- Go стал не просто популярным языком - он стал фундаментом, на котором работает вся современная облачная инфраструктура.
- Написано на Go (полностью):
  - Docker - контейнеризация, изменившая индустрию
  - Kubernetes - оркестрация контейнеров (Google → CNCF)
  - Terraform (HashiCorp) - Infrastructure as Code
  - Prometheus - мониторинг и алертинг
  - etcd - распределённое key-value хранилище (основа K8s)
  - CoreDNS - DNS-сервер Kubernetes
  - Consul, Vault, Nomad (HashiCorp)
  - CockroachDB - распределённая SQL-БД
  - Grafana Loki, Mimir, Tempo - observability-стек

Почему именно Go для инфраструктуры?

- Один статически скомпилированный бинарник → тривиальный Docker-образ (FROM scratch)
- Горутины → тысячи одновременных соединений без thread pool
- Быстрый старт → идеально для serverless и auto-scaling
- Cross-compilation → GOOS=linux GOARCH=arm64 go build — и готово

Нет наследования - и это не баг

- Проблема наследования (Rob Pike):
  - *Наследование продвигает важные архитектурные решения слишком рано - в фазу проектирования, до того как опыт может повлиять на них. Вы строите иерархию типов до того, как понимаете домен*

Нет исключений - и это осознанный выбор

- Исключения создают скрытый поток управления.
- Любая строка может бросить исключение. Вы не видите этого в сигнатуре функции (checked exceptions в Java - попытка решить это, которая создала свои проблемы)

```
// Java - где здесь может быть исключение? Где угодно.
User user = userService.findById(id);  // может бросить
String name = user.getName();          // может бросить NPE
logger.info("Found: " + name);        // может бросить
```

```
// Go: поток управления всегда виден
user, err := userService.FindByID(id)
if err != nil {
    return fmt.Errorf("finding user %s: %w", id, err)
}
// Если мы дошли сюда — user гарантированно валиден
```

Экспорт через регистр

```go
package user

type User struct {      // User — экспортировано (заглавная), видно извне пакета
    Name  string        // Name — экспортировано, часть публичного API
    email string        // email — НЕ экспортировано, внутренняя деталь
}

func New(name, email string) User { // New — экспортировано
    return User{
        Name:  name,
        email: email,
    }
}

func (u User) validate() error {    // validate — не экспортировано
    // ...
}
```

Переключение ментальной модели

- ООП-мышление (Java/C#):
  - 1. Определи объекты в домене → 2. Создай иерархию классов → 3. Добавь методы → 4. Данные подстроятся
- Go-мышление :
  - 1. Какие данные входят? → 2. Какие данные выходят? → 3. Какие трансформации нужны? → 4. Типы и функции следуют из данных

Переключение ментальной модели

```
// Java мышление классами
class OrderProcessor {
    private final OrderRepository repo;
    private final PaymentService payments;
    private final NotificationService notifications;

    public OrderResult process(Order order) {
        // Где данные? Разбросаны по 3 сервисам
        // Что тестировать? Нужны 3 мока
        // Что происходит? Скрыто за абстракциями
    }
}
```

Переключение ментальной модели

```go
// Go мышление данными и трансформациями
type Order struct {
    ID       string
    Items    []Item
    Customer Customer
    Total    Money
}

type ProcessedOrder struct {
    Order
    PaymentID string
    Status    string
    ProcessedAt time.Time
}
```

```go
// Каждая функция — чистая трансформация данных
func ValidateOrder(o Order) error { ... }
func CalculateTotal(items []Item) Money { ... }
func ProcessPayment(o Order) (string, error) { ... }
func FinalizeOrder(o Order, paymentID string) ProcessedOrder { ... }
```

Будут вопросы? Не стесняйтесь!